

Database Systems

Assignment02



Muzammil Siddiqui

23K-2001

BCS-5J

Q1:

A. Primary Entities and Attributes:

Entity Name	Attributes
FILM	<u>FilmID</u> , Title, Budget, ReleaseDate, DirectorName
ACTOR	<u>ActorID</u> , Name, Phone
PRODUCTION_COMPANY	<u>CompanyID</u> , Name

B. Main Relationships and Cardinalities:

Relationship	Description	Cardinality
FILM - ACTOR	A film can cast many actors, and an actor can act in many films.	M:N (Many-to-Many)
FILM - PRODUCTION_COMPANY	A film can be funded by one or more companies, and a company can fund many films.	M:N (Many-to-Many)
FILM - DIRECTOR	Each film has exactly one director, and the director's name is stored within the film record.	1:1 (One-to-One)

C. Associative Entity (Linking Table):

For the FILM-ACTOR Relationship (Casting):

Associative Entity:		CASTING
Attributes:		<i>FilmID</i> , <i>ActorID</i> , CharacterName
Primary Key (PK):		<i>FilmID</i> + <i>ActorID</i> (Composite Key)

This associative entity resolves the Many-to-Many relationship between FILM and ACTOR, while also storing the CharacterName that the actor plays in a specific film.

D. Explanation (DirectorName as Non-Entity):

The DirectorName should not be made a separate entity because:

- The scenario specifies that directors are not tracked separately (i.e., their personal details or multiple film involvements are not recorded).
- The director only exists in the context of a single film, with no independent attributes or relationships.
- Hence, it is more efficient to store the director's name as an attribute within the FILM entity, avoiding unnecessary complexity in the database model.

Q3:

A. Anomalies in the Unnormalized Table:

1. Insertion Anomaly:

- If a new school wants to register (e.g., “Hillview School, London”) but no teacher has worked there yet, we cannot insert the school into the table because NIN, tName, and hours would be NULL.
- Similarly, if we hire a new teacher but haven’t assigned them to a contract yet, we can’t insert their data without contract details.

2. Deletion Anomaly:

- If we delete all rows related to Park School, we would lose all information about the school itself, even though it might still exist in the system.
- Similarly, if Brown A finishes all teaching contracts, deleting their last record removes teacher details completely.

3. Update Anomaly:

- If Park School changes its city from Manchester to Salford, we would need to update multiple rows.
- If one row is missed, inconsistent data will appear. Some rows show Manchester, others Salford.

B. Normalization to 3NF:

We will normalize the data step-by-step from UNF > 1NF > 2NF > 3NF.

Step 1: UNF > 1NF (Remove Repeating Groups)

The table already has atomic values, so it's in 1NF.

1NF Table:

| NIN | contractNo | hours | tname | schoolID | schoolName | schoolCity |

Step 2: Identify Functional Dependencies

From the scenario and data, we can identify:

$NIN \rightarrow tName$

(Each teacher has one unique NIN.)

$schoolID \rightarrow schoolName, schoolCity$

(Each school ID uniquely identifies the school and city.)

$\{NIN, contractNo, schoolID\} \rightarrow hours$

(The combination of teacher, contract, and school determines the number of hours.)

Step 3: 1NF > 2NF (Remove Partial Dependencies)

In the current structure, non-key attributes (tName, schoolName, schoolCity) depend partially on part of a composite key.

To remove partial dependencies, we decompose into separate tables:

Table 1: TEACHER

NIN (PK)	tName
2103	Brown A
2198	Khan R
2205	Lewis H

FD: NIN > tName

Table 2: SCHOOL

schoolID (PK)	schoolName	schoolCity
S301	Park School	Manchester
S114	Green School	Birmingham

FD: schoolID > schoolName, schoolCity

Table 3: CONTRACT

contractNo (PK)
T9001

FD: contractNo > (no further dependency)

Table 4: TEACHING_ASSIGNMENT

NIN (FK)	contractNo (FK)	schoolID (FK)	hours
2103	T9001	S301	10
2198	T9001	S301	18
2205	T9001	S114	20
2103	T9001	S114	12

FD: {NIN, contractNo, schoolID} > hours

Step 4: 2NF > 3NF (Remove Transitive Dependencies)

In these decomposed tables, every non-key attribute depends only on the primary key. No transitive dependencies remain.

Therefore, the schema is now in Third Normal Form (3NF).

3NF Schema:

- TEACHER(NIN, tName)
- SCHOOL(schoolID, schoolName, schoolCity)
- CONTRACT(contractNo)
- TEACHING_ASSIGNMENT(NIN, contractNo, schoolID, hours)

Relationships:

- TEACHER 1:M TEACHING_ASSIGNMENT
- SCHOOL 1:M TEACHING_ASSIGNMENT
- CONTRACT 1:M TEACHING_ASSIGNMENT

Assumptions Taken:

- Each teacher (NIN) uniquely identifies one person.
- Each schoolID uniquely identifies a school and its location.
- contractNo uniquely identifies a teaching contract type.
- The combination (NIN, contractNo, schoolID) uniquely identifies each teaching record.

Q4:

A. Primary Key (PK)

From FD (4), the row is uniquely identified by the combination (FilmID, ActorID).

Ans: Primary Key: (FilmID, ActorID) (composite).

B. Highest Normal Form it violates (2NF or 3NF)?

The table is in 1NF (atomic values) but not in 2NF. There are partial dependencies, non-key attributes depend on part of the composite primary key rather than on the whole composite key.

Example of partial dependency (violation of 2NF):

- FilmID > FilmTitle, DirectorName

FilmTitle and DirectorName depend only on FilmID (part of the composite PK), not on the full (FilmID, ActorID).

- ActorID > ActorName, AgentID

ActorName and AgentID depend only on ActorID (part of the composite PK).

Because partial dependencies exist, the table violates 2NF (and therefore also violates 3NF until we remove these partials). So the highest NF violated is 2NF.

C. Decompose to Second Normal Form (2NF)

Goal: remove partial dependencies (i.e., attributes depending on FilmID alone or ActorID alone).

Decomposition (to remove partials):

- **FILM**

Attributes: FilmID (PK), FilmTitle, DirectorName

- removes FD FilmID > FilmTitle, DirectorName.

- **ACTOR**

Attributes: ActorID (PK), ActorName, AgentID

- removes FD ActorID > ActorName, AgentID.

- **FILM_CASTING** (the associative table)

Attributes: FilmID (FK), ActorID (FK)

Primary Key: (FilmID, ActorID)

- After removing attributes that depended on parts of the key, this table contains only the composite PK (and any attributes that truly depend on the full pair, but per FD (4) none of the remaining attributes do).

- **AGENT** (optional at this stage or left for 3NF)

We could either leave AgentPhone in ACTOR as AgentID only or create

AGENT now; but to remove transitive dependency (AgentID > AgentPhone)

we should make AGENT. This is usually done when moving to 3NF. For 2NF stage we can:

- leave AgentID in ACTOR and keep AgentPhone duplicated (still a problem),
- create AGENT now to remove transitive dependency (recommended).

Dependencies removed by the 2NF decomposition:

- FilmTitle, DirectorName no longer partially depend on composite key (moved to FILM).
- ActorName, AgentID no longer partially depend on composite key (moved to ACTOR).

D. Decompose to Third Normal Form (3NF)

Goal: remove transitive dependencies so every non-key attribute depends only on a key, the whole key, and nothing but the key.

From FDs we also have $\text{AgentID} \rightarrow \text{AgentPhone}$. In the earlier step ACTOR had AgentID; if we kept AgentPhone in ACTOR that would be a transitive dependency $\text{ActorID} \rightarrow \text{AgentID} \rightarrow \text{AgentPhone}$. So we must create an AGENT table.

Final 3NF tables (with PKs and FKs):

FILM:

- PK: FilmID
- Attributes: FilmTitle, DirectorName
- No FKs.

ACTOR:

- PK: ActorID
- Attributes: ActorName, AgentID (FK > AGENT.AgentID)
- FK: AgentID references AGENT(AgentID)

AGENT:

- PK: AgentID
- Attributes: AgentPhone

FILM_CASTING: (associative/linking table)

- PK: (FilmID, ActorID) (composite)
- FKs: FilmID > FILM(FilmID); ActorID > ACTOR(ActorID)

E. Redundancy Anomaly:

If AgentPhone is stored repeatedly for every film an actor appears in (i.e., not normalized) then changing the agent's phone number would require updating every row where that agent appears. This is an Update Anomaly.

Example (before normalization):

Suppose AgentID = A101 has AgentPhone = 555-1111 and appears in 10 FILM_CASTING rows (actor acted in 10 films). If the agent's phone changes to 555-2222:

- You must update all 10 rows to new phone.
- If you miss even one row, the database becomes inconsistent (some rows show old phone, some new).

That is the update anomaly caused by data redundancy.

Q5:

1) Functional Dependencies (FDs):

Assumptions:

- StudentID uniquely identifies a student and does not change.
- CourseID uniquely identifies a course (course code unique).
- Instructor is an entity (an instructor may teach multiple courses). The form only shows InstructorName. We assume real system will use an InstructorID. If instructor names are unique in this dataset you could use InstructorName as key, but to be robust we introduce InstructorID.
- A student enrolls in a course once per offering in this simplified example i.e., the enrollment is uniquely identified by (StudentID, CourseID) (no semester attribute shown). We keep EnrollmentDate as an attribute of that enrollment.
- CourseName and Credits depend on CourseID.
- Instructor (name or id) is assigned to course (i.e., CourseID > InstructorID).

Functional Dependencies:

1. StudentID > FirstName, LastName, Address
2. CourseID > CourseName, Credits, InstructorID

3. InstructorID > InstructorName

4. {StudentID, CourseID} > EnrollmentDate (enrollment record for that student in that course)

2) Normalization Process:

UNF > 1NF:

The form is already atomic (no repeating multi-valued cells in each row). So it is in 1NF.

1NF > 2NF:

- The repeating course rows show attributes that depend only on CourseID (CourseName, Credits, Instructor), not on the student.
- The composite key of the enrollment row (if we considered the whole row) would be (StudentID, CourseID) for enrollment records. Attributes CourseName, Credits, Instructor are partial dependencies on CourseID.
- Decompose to remove partials into separate STUDENT, COURSE, and ENROLLMENT tables.

2NF > 3NF:

- Course > InstructorID, and InstructorID > InstructorName is a transitive chain if InstructorName were kept in COURSE. So separate INSTRUCTOR table so Course refers to Instructor by ID.

- After moving InstructorName to INSTRUCTOR, no transitive dependency remains; each non-key attribute depends only on the key > 3NF.

Final 3NF Relations:

STUDENT(StudentID, FirstName, LastName, Address)

PK: StudentID

Alternate keys: none assumed (FullName is NOT guaranteed unique).

INSTRUCTOR(InstructorID, InstructorName)

PK: InstructorID

Alternate keys: (InstructorName) optionally, only if names are guaranteed unique (usually not).

COURSE(CourseID, CourseName, Credits, InstructorID)

PK: CourseID

FK: InstructorID > INSTRUCTOR(InstructorID)

Alternate keys: CourseName (only if guaranteed unique; usually CourseID is canonical).

ENROLLMENT(StudentID, CourseID, EnrollmentDate)

PK: (StudentID, CourseID) -- composite primary key

FKs:

StudentID > STUDENT(StudentID)

CourseID > COURSE(CourseID)

3) Primary, Alternate, & Foreign Keys:

STUDENT

- **Primary Key: StudentID**
- **Alternate Keys: none assumed**
- **Foreign Keys: none**

INSTRUCTOR

- **Primary Key: InstructorID**
- **Alternate Keys: optionally InstructorName only if unique**
- **Foreign Keys: none**

COURSE

- **Primary Key: CourseID**
- **Alternate Keys: optionally CourseName (if unique)**
- **Foreign Key: InstructorID > INSTRUCTOR(InstructorID)**

ENROLLMENT

- **Primary Key: composite (StudentID, CourseID) (or (StudentID, CourseID, Semester) if semesters exist)**

- Alternate Keys: none
- Foreign Keys:
 - StudentID > STUDENT(StudentID)
 - CourseID > COURSE(CourseID)